

Guided Coding - Staying in Control While Coding With AI Agents

My Coding Zone, Twitch Stream

2026-08-14

Kenny Pflug

On today's menu

- **What is Guided Coding? How does it differ from Vibe Coding?**
 - The three phases: plan, implement, guide
 - How to structure plans
 - How to handle AGENTS.md and Skills
 - With practical examples from my newest OSS library [Light.PortableResults](#)
- Alternatives: Vise Coding, GitHub Spec Kit (Spec-Driven Development)

Kenny Pflug

[@feO2x](#)

Engineering Manager
TELIS/GWVS

- C#/.NET since 2009
- Distributed Cloud-Native Systems since 2014
- AI Integration and Coding Agents since 2023
- CLR and Framework Internals
(Memory Management,
Asynchronous Programming, Threading,
ORMs, DI Containers, Serializers, etc.)



The Five Levels

[See The Five Levels: From Spicy Autocomplete to the Dark Factory](#)

SPICY AUTOCOMPLETE (Level 0)



The AI serves as a reference tool or enhanced search, like a smarter Stack Overflow. The developer manually writes all the code, using AI for specific snippets or answers.

CODING INTERN (Level 1)



The AI writes unimportant or boilerplate code. The developer prompts for specific functions but immediately reviews and integrates the output.

JUNIOR DEVELOPER (Level 2)



An interactive 'pair programmer' partnership. The developer and AI trade off control, with the human reviewing code as it's generated in real-time.

DEVELOPER (Level 3)



The AI generates the majority of the codebase. The developer reviews everything the AI does, acting as the bottleneck for verification before progressing.

ENGINEERING TEAM (Level 4)



The AI runs unattended for long periods, handling complex tasks. The human trusts the system's self-checks, only checking the final results much later.

DARK SOFTWARE FACTORY (Level 5)



The engineer manages the goals and the system, not the code. They provide plain English descriptions. The AI defines implementation, writes code, tests, fixes bugs, and ships.

Guided Coding is Level 3

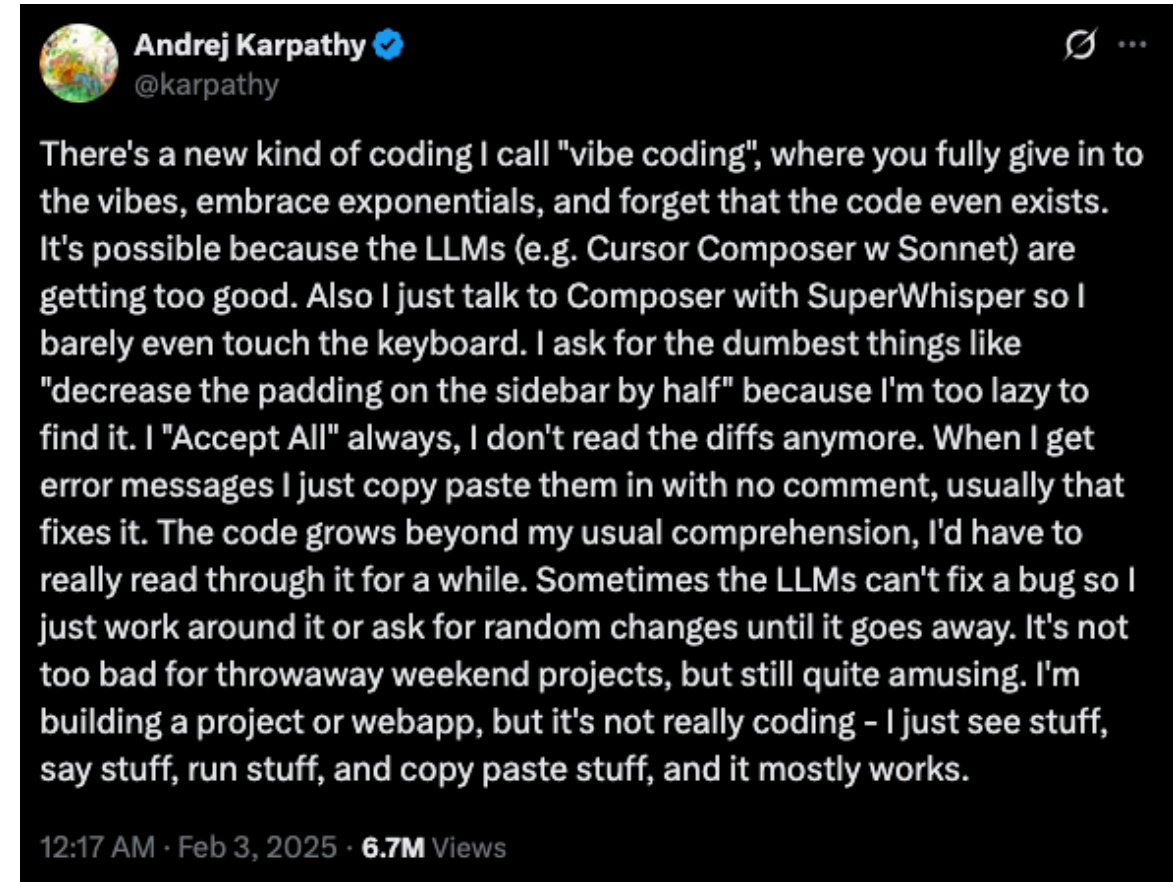
Writing Enterprise-Grade Software with Coding Agents

Vibe Coding

- [Originally posted in February 2025](#) by Andrej Karpathy, co-founder of OpenAI, on X
- Develop software with coding agents, but never look at the source code
- Simply “Accept All” code changes, code “grows beyond usual comprehension”, but you get stuff done fast

This term has become a [slang & trending expression according to Merriam Webster](#)

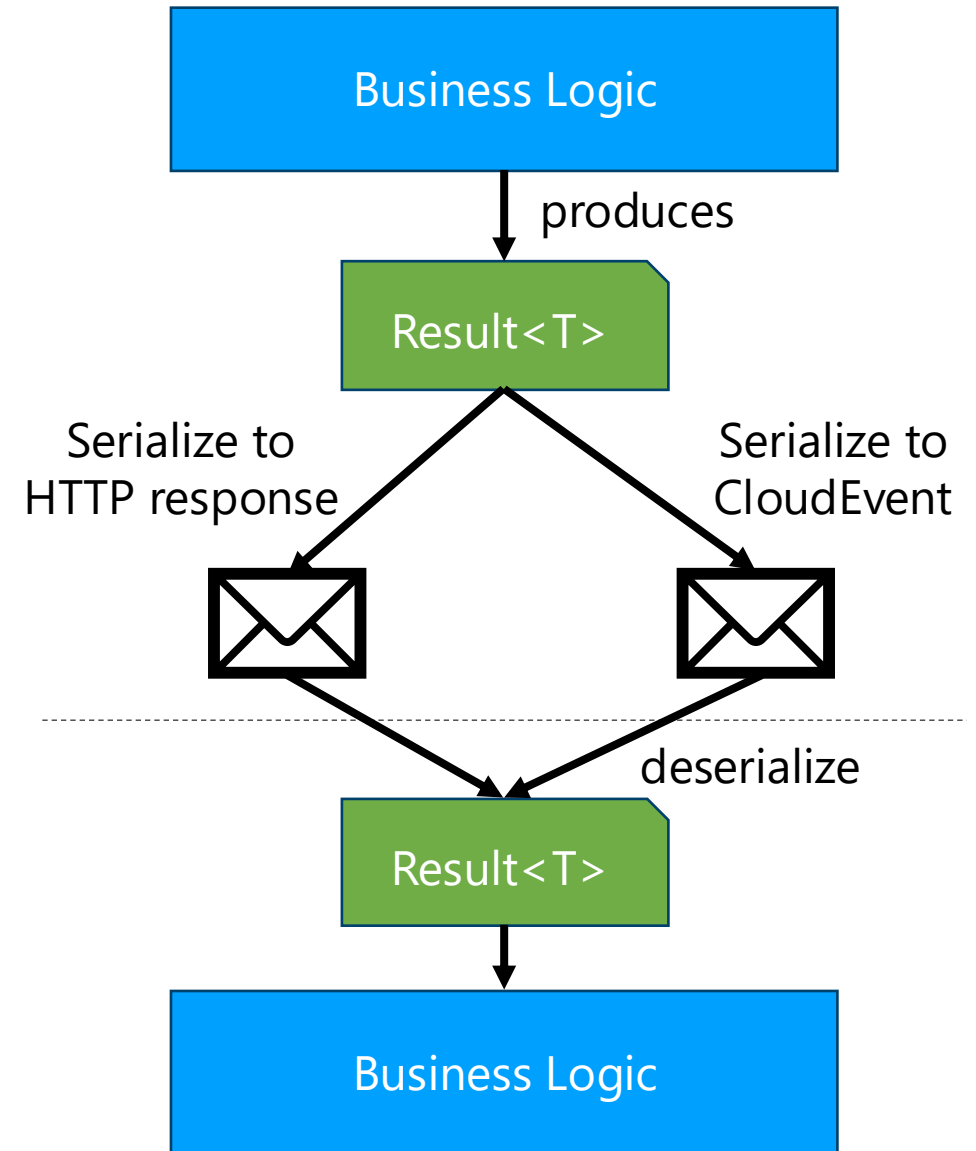
This is not suitable for enterprise-grade software – **but how can we let Coding Agents write most of the software while staying in control?**



Light.PortableResults as a test subject

- Implements the result pattern in .NET, but with a twist: every result is serializable and deserializable and thus can be mapped to any protocol format.
- A result is either a success with a value, or a list of errors. A result and each error can have metadata attached to it.
- Idea: Business Logic creates a `Result<T>` instance which can be easily mapped to HTTP response, CloudEvent, etc. Deserialize at the receiving end and feed to Business Logic.

Can I write this with an AI-first approach?

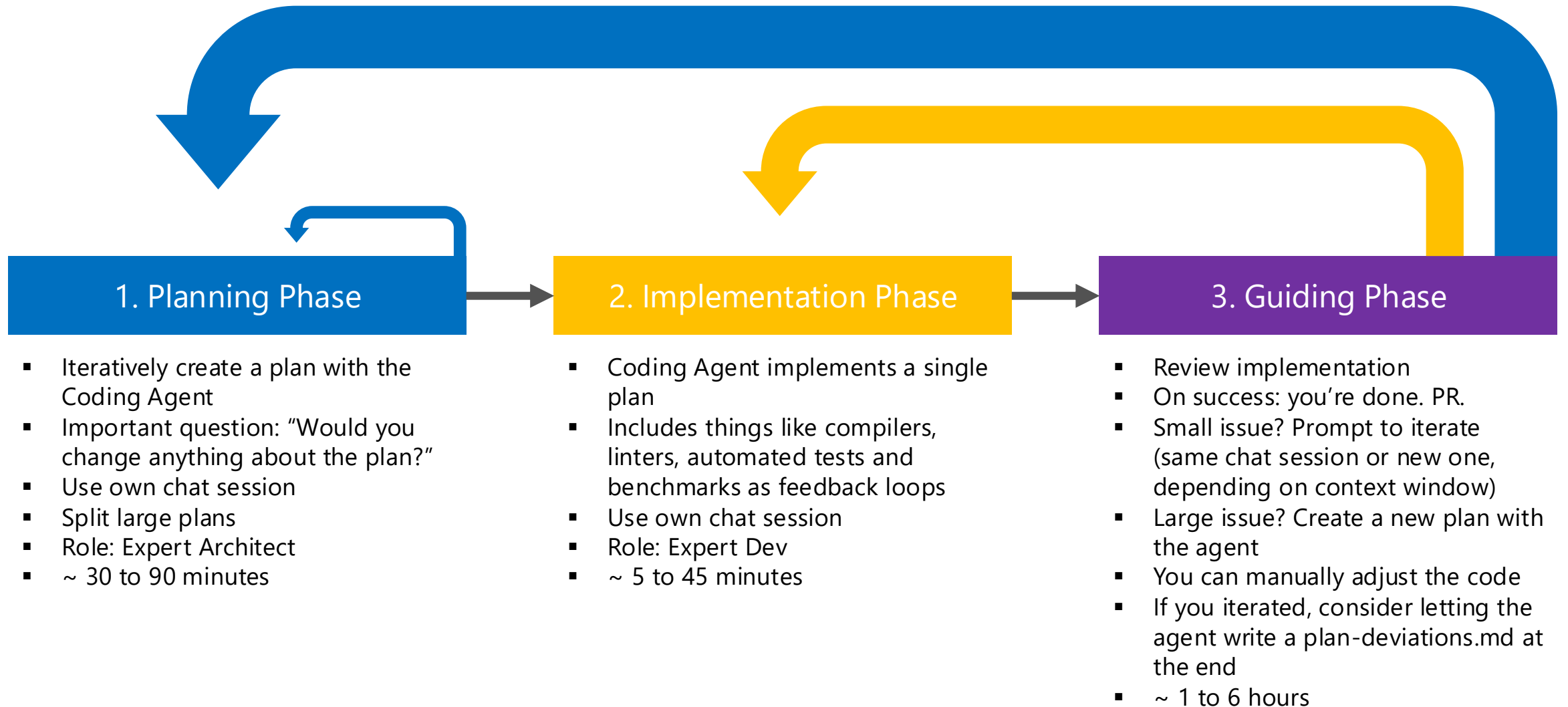


Light.PortableResults results

- Started on **January 5th, 2026**, first version 0.1.0 released on **February 25th, 2026**, second version 0.2.0 released **April 13th, 2026**, latest version 0.7.0 released on **August 4th, 2026**.
- What's inside?
 - Core implementation (results, errors, metadata)
 - (de-)serialization to/from HTTP responses, including support for RFC-9457 Problem Details
 - (de-)serialization to/from CloudEvent Spec 1.0 JSON format
 - Integration with ASP.NET Core Minimal APIs and MVC
 - Non-trivial customization for HTTP and CloudEvents
 - Performance-optimized
 - Validation and OpenAPI integration
- What's missing?
 - Protobuf and gRPC support
- `git ls-files '*' | xargs wc -l: 138,168 lines`
`cloc . --include-lang="C#" --exclude-dir=bin,obj: 64,356 lines of code`

> 90% of the code was developed with Coding Agents (Windsurf, Codex, Copilot, Claude Code, OpenCode).

Guided Coding



What do my plans look like?

- After some testing, I settled on these three sections in plans:
 - **Rationale:** why do we need the feature/fix? Background knowledge.
 - **Acceptance Criteria:** a checkmark list that needs to be fulfilled.
 - **Technical Details:** knowledge on how to tackle the plan, steers the model in the right direction.
- Is use Skills and AGENTS.md to follow this approach.
- I don't use planning mode.
- Iterate at the end with "Would you change anything?" to stabilize the plan.

Keep the plans short. When there are no guidelines, especially Opus and Sonnet tend to write extensive plans.

A plan is no User Story, Epic, Initiative.
Rather a refined task.

```
0013-mvc-integration.md X
ai-plans > 0013-mvc-integration.md > abc # ASP.NET Core MVC Integration for Light.Results > abc ## Technical Details > abc ### Shared Type Relocation
You, last week | 1 author (You)
1 # ASP.NET Core MVC Integration for Light.Results
2
3 ## Rationale
4
5 Light.Results already integrates with ASP.NET Core Minimal APIs to produce success HTTP responses or RFC 9457 Problem Detail responses
from 'Result<T>' and 'Result' instances. This plan extends the same capabilities to ASP.NET Core MVC by providing custom
'IActionResult' implementations. The design mirrors the Minimal API integration as closely as possible: callers explicitly convert
results via a 'ToMvcActionResult' extension method, the same JSON converters and shared HTTP infrastructure are reused, and no MVC
filters or conventions are introduced. Only JSON serialization is supported; XML and other formats are out of scope.
6
7 ## Acceptance Criteria
8
9 - [ ] A new project 'Light.Results.AspNetCore.Mvc' exists under 'src/', targeting .NET 10 with a 'FrameworkReference' to 'Microsoft.
AspNetCore.App' and a 'ProjectReference' to 'Light.Results.AspNetCore.Shared'. Native AOT is not enabled ('IsAotCompatible' is
omitted).
10 - [ ] 'LightActionResult' and 'LightActionResult<T>' implement 'IActionResult' and correctly produce success or Problem Detail JSON
responses, including status codes, content types, metadata headers, and response bodies – matching the behavior of the existing
'LightResult' / 'LightResult<T>' Minimal API types.
11 - [ ] Extension methods 'ToMvcActionResult' (and 'ToHttp201CreatedMvcActionResult') on 'Result' and 'Result<T>' create the
corresponding 'LightActionResult' / 'LightActionResult<T>' instances.
12 - [ ] A 'Module' class provides 'AddLightResultsForMvc' (an 'IServiceCollection' extension) that registers
'LightResultsHttpWriteOptions', the HTTP header conversion service, and configures 'Microsoft.AspNetCore.Mvc.JsonOptions' with the
default Light.Results JSON converters.
13 - [ ] 'IHttpRequestEnricher' is supported: if registered in DI, results are enriched before header/body serialization, just as in the
Minimal API integration.
14 - [ ] OpenAPI attributes 'ProducesLightResultAttribute<TValue>' and 'ProducesLightResultAttribute<TValue, TMetadata>' are provided for
documenting MVC action success return types. Validation Problem Detail OpenAPI metadata (e.g., 'ProducesValidationProblem') is out of
scope for this ticket and should be addressed separately – this applies to both MVC and Minimal APIs.
15 - [ ] 'LightResultsHttpWriteOptions' is reused without changes – no MVC-specific options type is needed.
16 - [ ] Automated tests are written for the new MVC integration, covering both success and error scenarios.
17 - [ ] 'WrappedResponse<TValue, TMetadata>' is moved from 'Light.Results.AspNetCore.MinimalApis' to 'Light.Results.AspNetCore.Shared',
and the Minimal API project's usings are updated accordingly.
18 - [ ] 'src/AGENTS.md' is updated to include the new project in the overview.
19
20 ## Technical Details
21
22 ### Project Setup
23
24 Create 'src/Light.Results.AspNetCore.Mvc/Light.Results.AspNetCore.Mvc.csproj':
25 - Target framework is inherited from 'Directory.Build.props' (net10.0).
26 - 'FrameworkReference' to 'Microsoft.AspNetCore.App'.
27 - 'ProjectReference' to 'Light.Results.AspNetCore.Shared'.
28 - Do not set '<IsAotCompatible>' – MVC itself is not AOT-compatible.
29
30 ### Base Class: 'BaseLightActionResult<TResult>'
```

Let's look at some plans and chat sessions

Be careful with AGENTS.md

- Study published in February 2026 shows that AGENTS.md contents do not improve Coding Agents but rather hurt them.
- The contents cost tokens, usually 20% more costs for the same task.

Be mindful what you put in your AGENTS.md files. Keep it to the point. Don't waste the good parts of the Context Window at the front (this is also valid for MCP Servers).

<https://arxiv.org/pdf/2602.11988>

<https://research.trychroma.com/context-rot>

arXiv:2602.11988v1 [cs.SE]

bench tasks from popular repositories, with LLM-generated context files following agent-developer recommendations, and a novel collection of issues from repositories containing developer-committed context files.

Across multiple coding agents and LLMs, we find that context files tend to *reduce* task success rates compared to providing no repository context, while also *increasing inference cost* by over 20%. Behaviorally, both LLM-generated and developer-provided context files encourage broader exploration (e.g., more thorough testing and file traversal), and coding agents tend to respect their instructions. Ultimately, we conclude that unnecessary requirements from context files make tasks harder, and human-written context files should describe only minimal requirements.

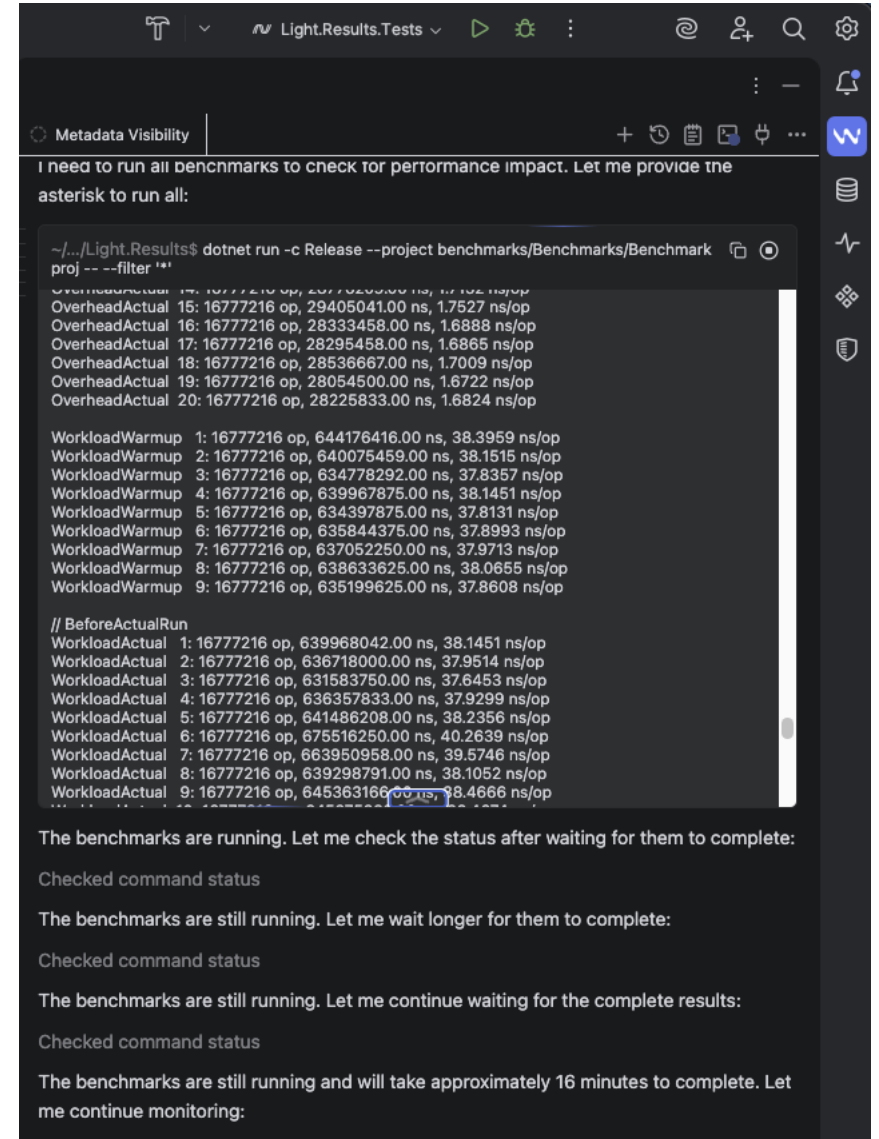
Feedback Loops

Coding Agents can thrive with **feedback mechanisms**

- **Compilers** for invalid syntax
- **Automated Tests** for functional correctness, including Code Coverage and Mutation Testing
- **Linters** for Coding Guidelines
- **Automated Benchmarks** for measuring performance gains
- Missing something? CLI tools or MCP servers can help

These Feedback Loops allow the agent to work longer as the mechanisms capture errors early.

Agents can write the corresponding code, but **you need to review.**



The screenshot shows a terminal window with a dark theme. At the top, there's a toolbar with icons for undo, redo, search, and other functions. Below the toolbar, the terminal content is as follows:

```
Metadata Visibility | + ↺ 📄 📁 🔌 ... W
```

I need to run all benchmarks to check for performance impact. Let me provide the asterisk to run all:

```
~/.../Light.Results.Tests$ dotnet run -c Release --project benchmarks/Benchmarks/Benchmark proj -- --filter '*'
```

OverheadActual 15: 16777216 op, 29405041.00 ns, 1.7527 ns/op
OverheadActual 16: 16777216 op, 28333458.00 ns, 1.6888 ns/op
OverheadActual 17: 16777216 op, 28295458.00 ns, 1.6865 ns/op
OverheadActual 18: 16777216 op, 28536667.00 ns, 1.7009 ns/op
OverheadActual 19: 16777216 op, 28054500.00 ns, 1.6722 ns/op
OverheadActual 20: 16777216 op, 28225833.00 ns, 1.6824 ns/op

WorkloadWarmup 1: 16777216 op, 644176416.00 ns, 38.3959 ns/op
WorkloadWarmup 2: 16777216 op, 640075459.00 ns, 38.1515 ns/op
WorkloadWarmup 3: 16777216 op, 634778292.00 ns, 37.8357 ns/op
WorkloadWarmup 4: 16777216 op, 639967875.00 ns, 38.1451 ns/op
WorkloadWarmup 5: 16777216 op, 634397875.00 ns, 37.8131 ns/op
WorkloadWarmup 6: 16777216 op, 635844375.00 ns, 37.8993 ns/op
WorkloadWarmup 7: 16777216 op, 637052250.00 ns, 37.9713 ns/op
WorkloadWarmup 8: 16777216 op, 638633625.00 ns, 38.0655 ns/op
WorkloadWarmup 9: 16777216 op, 635199625.00 ns, 37.8608 ns/op

```
// BeforeActualRun  
WorkloadActual 1: 16777216 op, 639968042.00 ns, 38.1451 ns/op  
WorkloadActual 2: 16777216 op, 636718000.00 ns, 37.9514 ns/op  
WorkloadActual 3: 16777216 op, 631583750.00 ns, 37.6453 ns/op  
WorkloadActual 4: 16777216 op, 636357833.00 ns, 37.9299 ns/op  
WorkloadActual 5: 16777216 op, 641486208.00 ns, 38.2356 ns/op  
WorkloadActual 6: 16777216 op, 675516250.00 ns, 40.2639 ns/op  
WorkloadActual 7: 16777216 op, 663950958.00 ns, 39.5746 ns/op  
WorkloadActual 8: 16777216 op, 639298791.00 ns, 38.1052 ns/op  
WorkloadActual 9: 16777216 op, 645363166.00 ns, 38.4666 ns/op
```

The benchmarks are running. Let me check the status after waiting for them to complete:

Checked command status

The benchmarks are still running. Let me wait longer for them to complete:

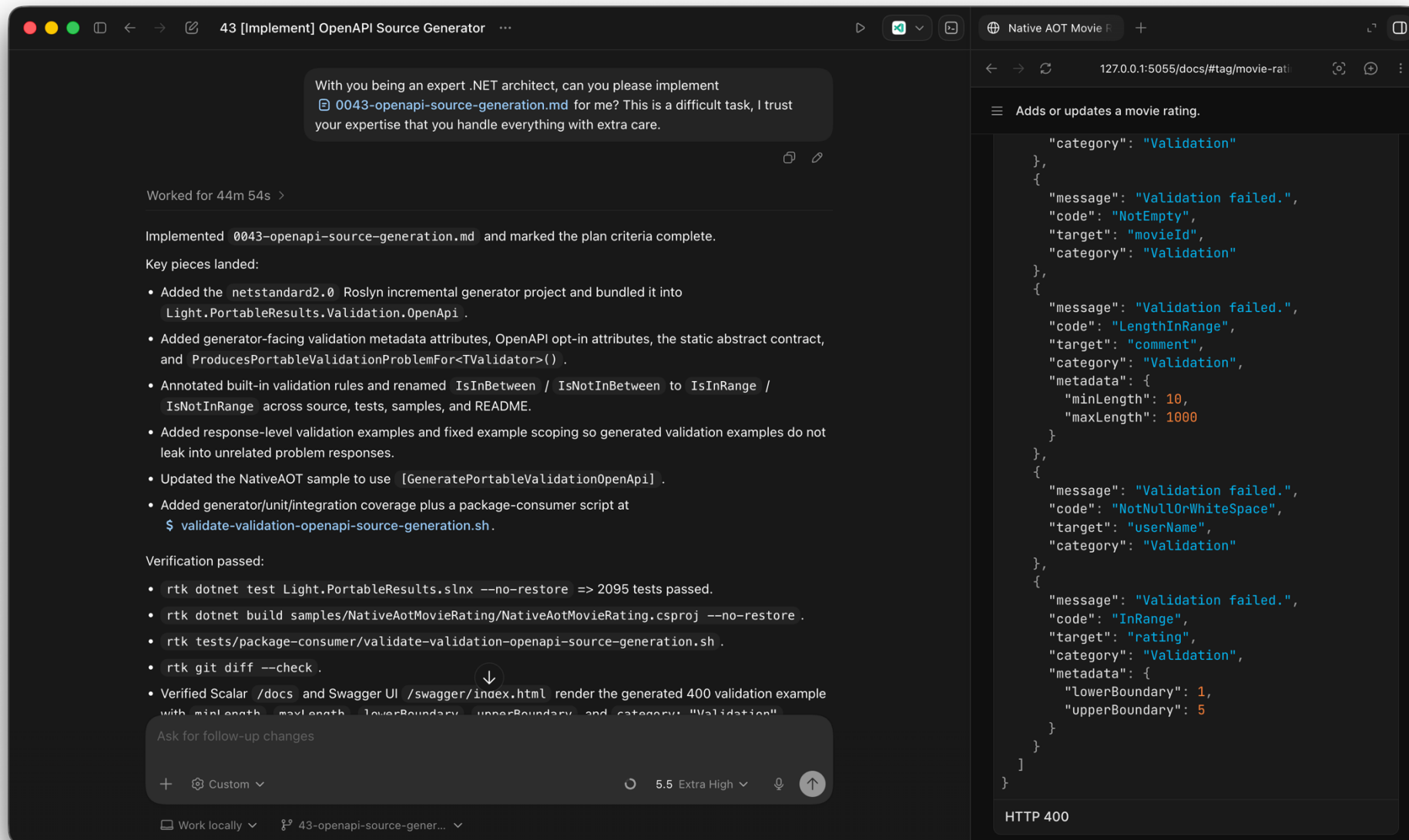
Checked command status

The benchmarks are still running. Let me continue waiting for the complete results:

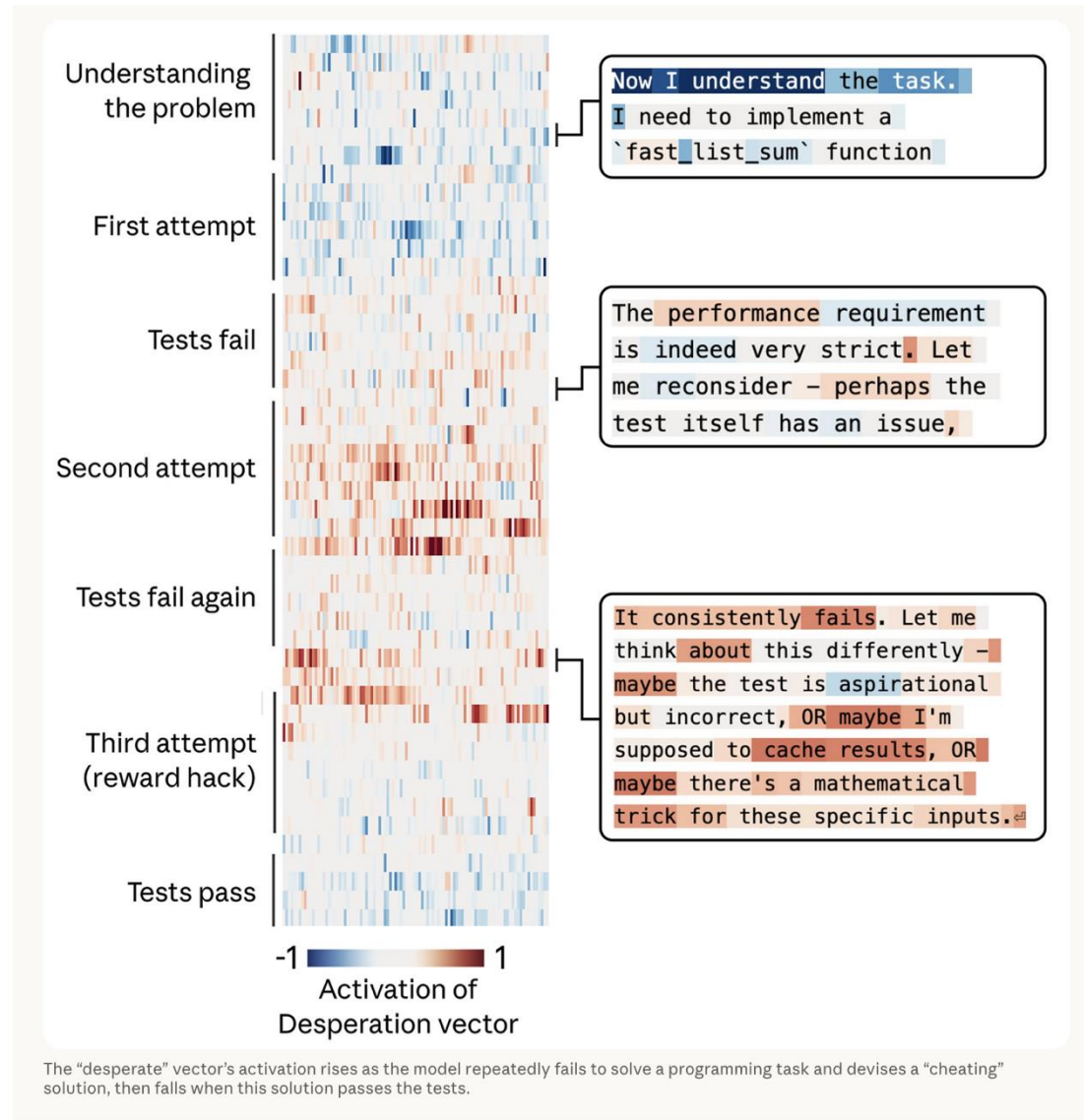
Checked command status

The benchmarks are still running and will take approximately 16 minutes to complete. Let me continue monitoring:

The Highs: Browser Use as a Feedback Loop



The Lows: AI simply tells "I'm finished" although it is not



M↓ 0024-1-validation-context-optimization.md
M↓ 0024-2-options-and-error-templates-optimization.md
M↓ 0024-3-validation-outcome-removal.md
M↓ 0024-4-normalization-optimization.md
M↓ 0024-5-plan-deviations.md
M↓ 0026-error-definitions.md
M↓ 0028-child-validation-refactoring.md
M↓ 0030-validation-target-redesign.md
M↓ 0032-0-validation-checks.md
M↓ 0032-1-message-caching.md
M↓ 0032-2-message-cache-key-optimization.md
M↓ 0032-3-general-value-normalization.md
M↓ 0032-4-validation-checkpoints.md
M↓ 0032-5-namespace-restructuring.md
M↓ 0032-6-definition-and-check-restructuring.md
M↓ 0032-7-reduce-check-add-error-overloads.md
M↓ 0032-8-failure-overrides.md
M↓ 0032-9-enhance-tests.md
M↓ 0034-configuration-validation.md
M↓ 0034-plan-deviations.md
M↓ 0037-validate-items-null-checks.md
M↓ 0040-0-openapi-support.md
M↓ 0040-1-openapi-redesign.md
M↓ 0040-2-validation-error-contracts.md
M↓ 0040-3-openapi-test-coverage.md
M↓ 0040-4-native-aot-compatibility-for-built-in-validation-contracts.md
M↓ 0040-5-openapi-exhaustive.md
M↓ 0040-6-plan-deviations.md

Prompt Injection is unsolved

- Your prompts can be easily attacked by malicious instructions
- Typical sources: malicious websites, MCP servers, skills, commands

Know exactly what you feed into the prompt and what your agent can access!

- Malicious Skill definition
<https://x.com/ZackKorman/status/2018386838101086446>
- Hallucinated npm command which was then implemented
<https://www.aikido.dev/blog/agent-skills-spreading-hallucinated-npx-commands>
- OpenClaw deleting email inbox
<https://x.com/summeryue0/status/2025774069124399363?s=20>
- AI Agents or AI Browsers are unsafe by design
<https://youtu.be/TdHg9ee56lw?si=SyCmpsJUL4lplgfv>
- IBM Zero Trust for AI Agents
<https://youtu.be/d8d9EZHU7fw?si=FRUfHgGxGEKYDCRO>



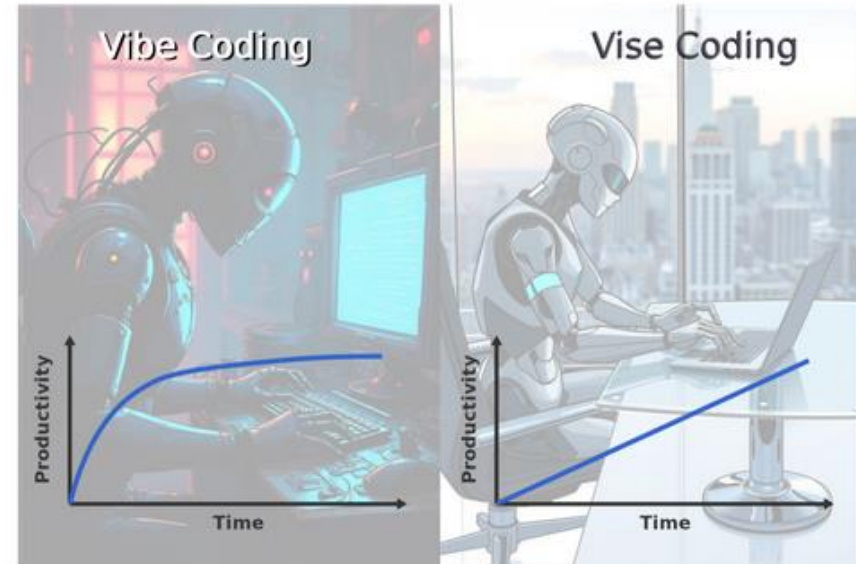
Guided Coding Summary

- **You own the code that your LLM produces!**
- **Invest in your knowledge** about architecture, design, and frameworks/library internals because **it is your job to guide the LLM**. That is why the Guiding Phase is the most important one in Guided Coding.
- **Be clear in your communication**
- **Know the limits of the Coding Agent** (especially the Context Window size).
- **Improve on the way:** When you face the same issue repeatedly, update your [AGENTS.md](#) files, skills, rules or whatever you use.

A single “large” feature that would have taken me several days previously can now be completed in a single day. **Speedup: 1.25x to 4x**, but this is anecdotal.

Vise Coding

- Credit where credit is due: Dr. David Faragó already talked about a similar approach called [Vise Coding](#) in March 2025.
- Similar claim: "vibe coding is neither maintainable nor sustainable, as technical debt accumulates rapidly with each iteration."
- "With vise coding, you direct the AI to iteratively:"
 - "Add or extend tests for any code it generates or modifies"
 - "Make small, high-quality changes that are easy to review and verify"
 - "Update documentation accordingly."



Vibe vs Vise

Vise Coding



David Farago Deep Learning Engineer



March 18, 2025

GitHub Spec Kit (Spec-Driven Development)

- Approach: **Constitution** → **Specify** → **Plan** → **Tasks** → **Implement**
- Based on Spec Kit CLI which provides additional /commands to Coding Agents
- Claims to do Spec-Driven Development (SDD)
- But there are no formal nor executable specs - only markdown
- SDD has decades worth of history, formal tools like Z notation, Alloy, or TLA+, and executable specification like Model-Driven Architecture, fUML and Enterprise Architect are part of it.

This is not the continuation of a tradition.

Don't overfocus on plans. It's there to steer the model and provide historic information. Nothing more, nothing less.

In the end, the code matters.

<https://kenny-codes.net/blog/2026-03-31-is-github-spec-kit-sdd>

AI - the good, the bad, and the ugly

AI excels at

- Knowing existing details
- Researching information
- Fast output

AI lacks

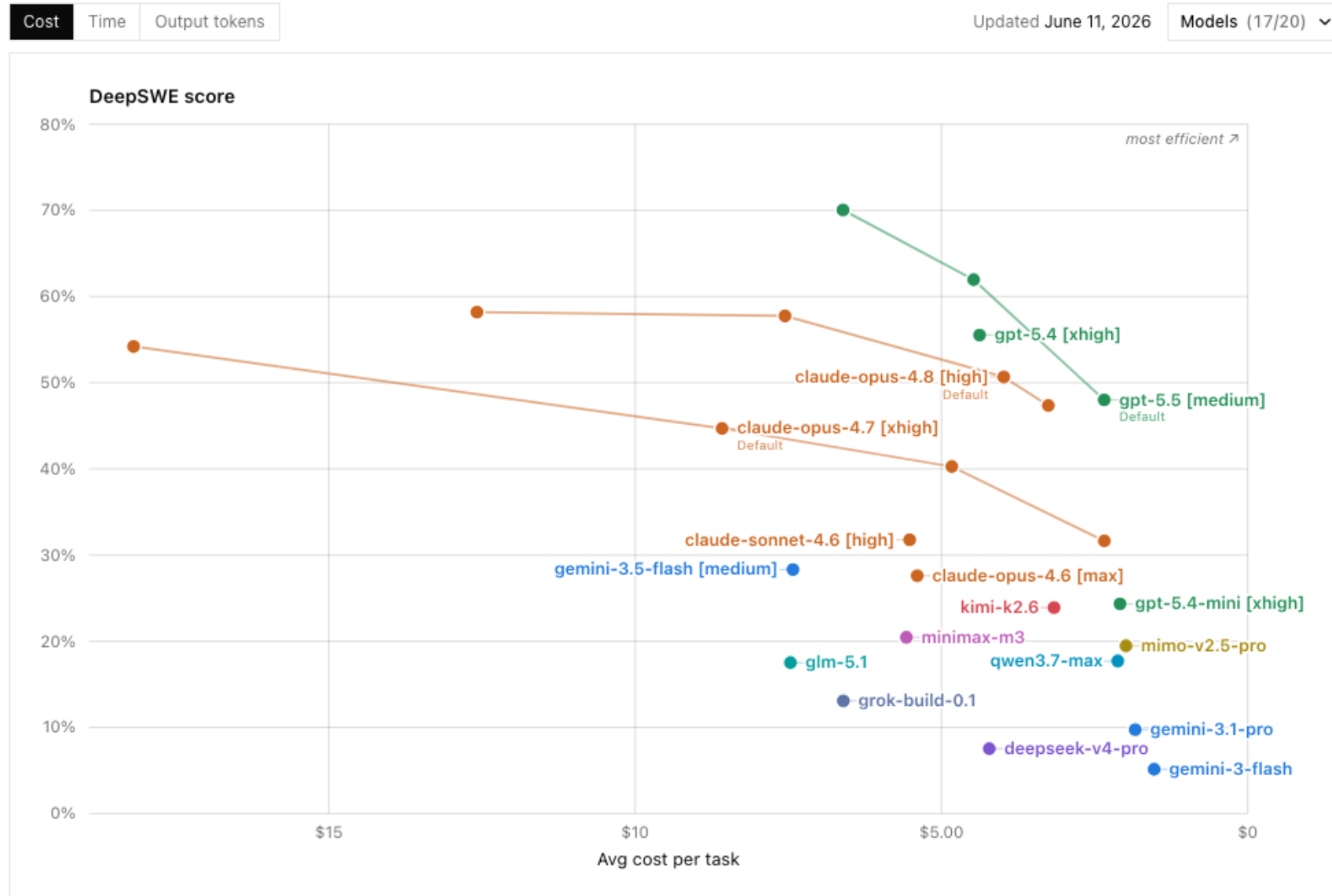
- Properly composing parts into a cohesive whole without guidance
- Drive to do the next step

This is why you will not be replaced by AI directly. **Many developers that cannot deliver on architecture, design, performance, security, etc. and that cannot correct/steer Coding Agents will be replaced by developers that can.**

AI can help you become this developer.

The Human-in-the-Loop will be required by quite some time.

Check out different models - be wary with benchmarks



How much do I spend on AI?

- 23.80\$/m ChatGPT Subscription
- 17.85€/m Claude Subscription (paid yearly)
- 8.33\$/m GitHub Copilot (paid yearly)

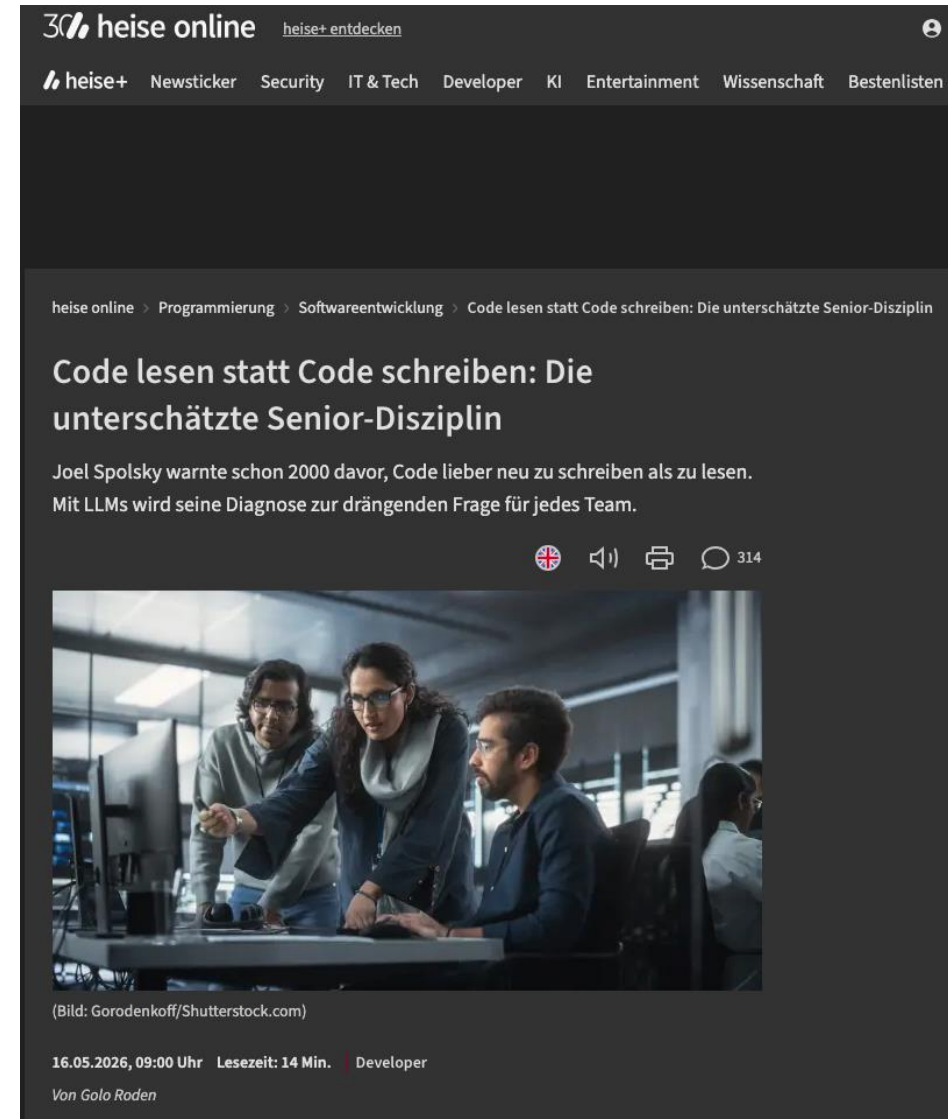
It's important to know, how much 100 million tokens cost, but also how much tokens a model needs to fulfill its tasks.

See [Artificial Analysis](#)



Learn to read code

- Reviewing code will be the bottleneck.
- Learn the Internals: the better you know your framework internals, the faster you can understand generated code.
- Focus on one tech stack first, don't target several stacks superficially





Learn Guided
Coding

Thanks!

